

COMPUTER SCIENCE DEPARTMENT

Total Marks: 3

Obtained Marks: _____

Parallel and Distributed Computing

Assignment # 3a

Submitted To: Rana Faisal Hayat

Student Name: Abdul Moez Naeem Malik

Reg Number: 2212369

Assignment #03 :-

Question #01: A smart-weather monitoring uses-oriented communication based on the publish-subscribe pattern. You must implement a simple-weather-alert broadcasting service using ZeroMQ in Python. The system consist of:

- A weather publisher (server) that periodically publish temperature updates.
- A weather subscriber (client) that subscribes only to message with topic "WEATHER" and displays the received temperature of information.

I) ZeroMQ Publisher (Server):

```
import time
import zmq
HOST = "127.0.0.1"
PORT = "5555"

def run_publisher():
    context = zmq.Context()
    publisher = context.socket(zmq.PUB)
    connection_address = f"tcp://{HOST}:{PORT}"
    print(f"Weather Publisher started. Binding to {connection_address}...")
    try:
        while True:
            topic = "WEATHER"
            body = "Islamabad 32°C"
            publisher.send_multipart([topic.encode("utf-8"), body.encode("utf-8")])
            print(f"Published: [{topic}] {body}")
            time.sleep(3)
    except KeyboardInterrupt:
        print("In Publisher Stopped.")
    finally:
        publisher.close()
        context.term()
```

```
if __name__ == "__main__":
```

```
    run_publisher()
```

11) ZeroMQ Subscriber (Client):

```
import zmq
```

```
HOST = '127.0.0.1'
```

```
PORT = '5555'
```

```
def run_subscriber():
```

```
    context = zmq.Context()
```

```
    subscriber = context.socket(zmq.SUB)
```

```
    connection_address = f"tcp://{HOST}:{PORT}"
```

```
    subscriber.connect(connection_address)
```

```
    subscriber.setsockopt_string(zmq.SUBSCRIBE, "WEATHER")
```

```
    print(f"Weather Subscriber started. Connected to {connection_address}...")
```

```
    print("Waiting for updates...\n")
```

```
    message_received = 0
```

```
    while message_received < 5:
```

```
        topic, body = subscriber.recv_multipart()
```

```
        topic_str = topic.decode("utf-8")
```

```
        topic_str = topic.decode("utf-8")
```

```
        print(f"Received update {message_received + 1}: [{topic_str}] {body_str}")
```

```
        message_received += 1
```

```
    print("\nReceived 5 updates. Terminating subscriber.")
```

```
    subscriber.close()
```

```
    context.term()
```

```
if __name__ == "__main__":
```

```
    run_subscriber()
```

Question #02: Two mpi4py code snippets for communication between process 0 and process 1 in a distributed image-processing system. One process generate image metadata, while the other process receives and process it.

1. Program Description and Roles:

- Program (A) utilizes blocking point-to-point communication.
- Program (B) utilizes non-blocking (asynchronous) point to point communication.

Roles: In both programs, Process 0 (rank 0) is the sender that generates and transmits the image-info dictionary metadata, while Process 1 (rank 1) is the receiver that fetches and processes it.

2. Blocking vs. Non-Blocking Communication:

The core difference lies in whether the execution thread waits for the communication operation to complete before moving to the next line of the code.

Blocking: (`comm.send`/`comm.recv`): In program A, when Process 0 calls `comm.send()`, execution halts until the summary is safely copied into a system buffer or delivered to the receiver. Similarly 1's `comm.recv()` completely halts execution until it successfully receives the data.

Non-Blocking: (`comm.isend`/`comm.irecv`): In Program B, `comm.isend()` initiates the send operation and immediately return a request handle (`req`), allowing the program to execute subsequent lines of code while data transfer happens in the background. The actual synchronization/completion check is explicitly deferred to `req.wait()`.

3. Impact of Removing `req.wait()` from the Sender in Program (B):

If `req.wait()` is removed from Process 0, the program will continue execution immediately after `comm.isend()`. This introduced two severe risks:

• Data Corruption / Race Conditions: If process 0, modifies the image-info dictionary immediately after the `comm.isend()` line, the MPI Library might read the modified or corrupted data while it is transferring it in the background. The receiver would end up with corrupted or incorrect metadata.

Resource Leaks:

The underlying MPI communication structures and memory buffers may not be properly released or finalized, which can degrade performance or crash the program over continuous iterations.